

UDF - CENTRO UNIVERSITARIO

RELATORIO DE ATIVIDADES PRATICAS

ARRAYS, MATRIZES E ALGORITMOS DE ORDENACAO E BUSCA

Versao 2 - Bubble Sort e Quick Sort

Curso	Bacharelado em Engenharia de Software
Disciplina	Estrutura de Dados II
Professora	Karla Roberta
Estudantes	Pietro Vianna da Rocha Joao Pedro Alves de Sousa
Sistemas utilizados	Python 3.12, Google Colab, Matplotlib e ReportLab
Data da entrega	Setembro / 2026

Brasilia, DF
2026

1. INTRODUCAO

Esta segunda versao da atividade investiga algoritmos de ordenacao e busca, relacionando tamanho da entrada, numero de operacoes, complexidade e eficiencia. A comparacao principal e feita pela quantidade de comparacoes, trocas e movimentacoes, porque o tempo de execucao tambem depende do computador utilizado.

A atividade foi organizada em seis partes: pesquisa teorica, experimento de ordenacao, busca sequencial em matrizes, dois estudos praticos com arrays e matrizes e uma analise final dos resultados.

2. PARTE 1 - PESQUISA: BUBBLE SORT E QUICK SORT

2.1. Bubble Sort

Como funciona: percorre o vetor repetidamente e compara pares de elementos adjacentes. Sempre que o elemento da esquerda e maior que o da direita, os dois sao trocados.

Logica de ordenacao: dois loops controlam as passagens e as comparacoes. Nesta implementacao, uma flag detecta quando nenhuma troca ocorre e encerra o algoritmo antecipadamente.

Complexidade: melhor caso $O(n)$, com parada antecipada; caso medio $O(n^2)$; pior caso $O(n^2)$.

Vantagens: implementacao simples, facil de compreender, in-place e estavel.

Limitacoes: grande quantidade de comparacoes e trocas em vetores maiores; baixa escalabilidade.

Uso adequado: conjuntos pequenos, quase ordenados ou situacoes didaticas.

Uso nao recomendado: grandes volumes de dados ou aplicacoes com exigencia de alto desempenho.

2.2. Quick Sort

Como funciona: utiliza a estrategia dividir para conquistar. Escolhe um pivo, particiona o vetor em torno dele e aplica o mesmo processo recursivamente nas duas partes.

Logica de ordenacao: a funcao de particionamento compara os elementos com o pivo e reorganiza o subvetor. Na implementacao utilizada, o ultimo elemento e escolhido como pivo.

Complexidade: melhor caso $O(n \log n)$; caso medio $O(n \log n)$; pior caso $O(n^2)$, quando as particoes ficam muito desbalanceadas.

Vantagens: excelente desempenho medio, boa escalabilidade e pouco uso de memoria auxiliar.

Limitacoes: nao e estavel na forma classica e pode degradar para $O(n^2)$ com escolhas ruins de pivo.

Uso adequado: conjuntos grandes e aplicacoes que exigem bom desempenho medio.

Uso nao recomendado: quando estabilidade e obrigatoria ou quando nao se pode tolerar o pior caso quadratico sem uma estrategia adequada de pivo.

2.3. Tabela comparativa

Caracteristica	Bubble Sort	Quick Sort
Principio	Comparacao e troca de pares adjacentes	Dividir para conquistar; particionamento por pivo
Melhor caso	$O(n)$, com flag de parada	$O(n \log n)$
Caso medio	$O(n^2)$	$O(n \log n)$
Pior caso	$O(n^2)$	$O(n^2)$
Uso de memoria	$O(1)$ - in-place	$O(\log n)$ em media - pilha recursiva
Vantagem principal	Simplicidade	Bom desempenho medio
Limitacao principal	Ineficiente para grandes vetores	Pode degradar com pivo ruim
Aplicacao recomendada	Dados pequenos/quase ordenados	Dados maiores e bom desempenho medio

3. PARTE 2 - EXPERIMENTO DE ORDENACAO

Os dois algoritmos recebem copias do mesmo vetor. Dessa forma, a comparacao mede o comportamento dos algoritmos e nao uma diferenca nos dados de entrada.

3.1. Codigo - Bubble Sort

```
def bubble_sort(arr):
    a = arr[:]
    n = len(a)
    comparisons = swaps = 0

    for i in range(n - 1):
        trocou = False
        for j in range(n - 1 - i):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1
                trocou = True
        if not trocou:
            break

    return a, comparisons, swaps
```

3.2. Codigo - Quick Sort

```
def quick_sort(arr):
    a = arr[:]
    comparisons = movements = 0

    def partition(lo, hi):
        nonlocal comparisons, movements
        pivot = a[hi]
        i = lo - 1
        for j in range(lo, hi):
            comparisons += 1
            if a[j] <= pivot:
                i += 1
                a[i], a[j] = a[j], a[i]
                movements += 1
        a[i + 1], a[hi] = a[hi], a[i + 1]
        movements += 1
        return i + 1

    def qs(lo, hi):
        if lo < hi:
            p = partition(lo, hi)
            qs(lo, p - 1)
            qs(p + 1, hi)

    qs(0, len(a) - 1)
    return a, comparisons, movements
```

3.3. Execucao comparativa

```
import random

random.seed(42)

for n in [10, 20, 1000]:
    original = [random.randint(0, 100000) for _ in range(n)]

    copia_bubble = original[:]
    copia_quick = original[:]

    _, b_cmp, b_swap = bubble_sort(copia_bubble)
    _, q_cmp, q_mov = quick_sort(copia_quick)

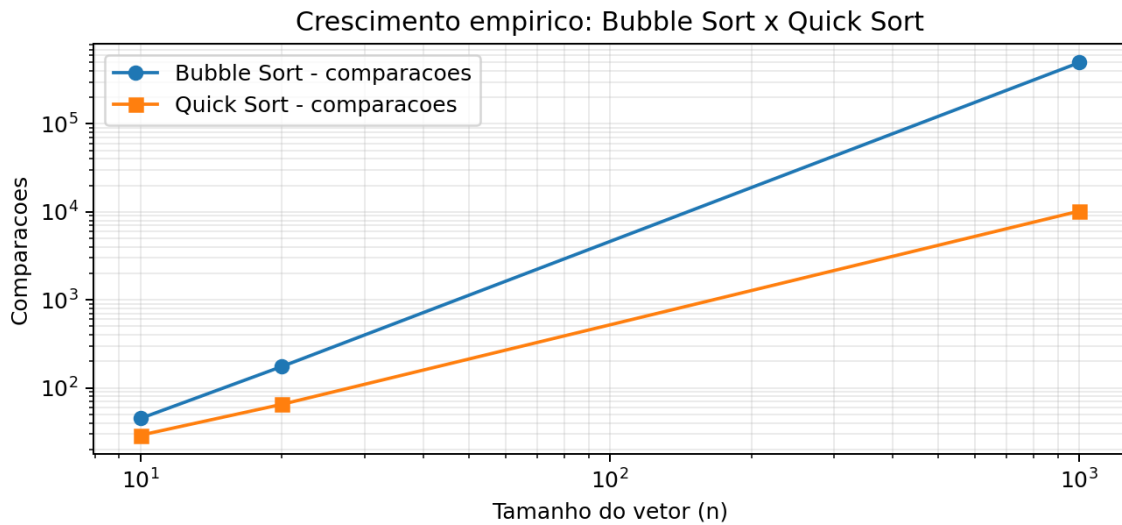
    print(n, b_cmp, b_swap, q_cmp, q_mov)
```

3.4. Resultados obtidos

Tamanho	Bubble - Comparacoes	Bubble - Trocas	Quick - Comparacoes	Quick - Movimentacoes
10	45	26	29	15
20	175	87	65	41
1.000	498.324	243.438	10.218	6.231

Os testes foram reproduzidos com **random.seed(42)**, garantindo os mesmos dados em novas execucoes do programa.

3.5. Analise dos resultados



a) Qual algoritmo realizou menos operacoes para 10 elementos?

O Quick Sort: 29 comparacoes e 15 movimentacoes, enquanto o Bubble Sort realizou 45 comparacoes e 26 trocas.

b) O comportamento permaneceu igual para 20 elementos?

Sim. O Quick Sort continuou realizando menos operacoes e a diferenca aumentou.

c) O que aconteceu quando o tamanho aumentou para 1.000 elementos?

A diferenca ficou muito maior: o Bubble Sort realizou 498.324 comparacoes, enquanto o Quick Sort realizou 10.218.

d) Qual algoritmo apresentou maior crescimento?

O Bubble Sort, por apresentar crescimento quadratico $O(n^2)$.

e) Os resultados sao coerentes com a teoria?

Sim. O comportamento observado acompanha as complexidades $O(n^2)$ e $O(n \log n)$.

f) Quando escolher Bubble Sort?

Em vetores pequenos, quase ordenados ou em atividades didaticas.

g) Quando escolher Quick Sort?

Em conjuntos maiores, quando se deseja melhor desempenho medio e uma estrategia adequada de pivo.

4. PARTE 3 - INVESTIGACAO DE BUSCA EM MATRIZES

A busca sequencial percorre cada linha e, dentro dela, cada coluna. A funcao informa se o valor foi encontrado, a linha, a coluna e a quantidade de comparacoes.

```
def busca_sequencial_matriz(matriz, valor):  
    comparacoes = 0  
  
    for i in range(len(matriz)):  
        for j in range(len(matriz[0])):  
            comparacoes += 1  
            if matriz[i][j] == valor:  
                return True, i, j, comparacoes  
  
    return False, -1, -1, comparacoes
```

4.1. Resultados

Matriz	Nº de elementos	Busca no inicio	Busca no final	Valor inexistente
2 x 2	4	1	4	4
10 x 10	100	1	100	100
100 x 100	10.000	1	10.000	10.000

a) Por que encontrar um elemento no inicio exige menos operacoes?

Porque a busca termina assim que encontra a primeira correspondencia.

b) O que acontece quando o elemento nao existe?

Todas as posicoes precisam ser verificadas antes de concluir que o valor nao esta presente.

c) Qual e o pior caso?

Quando o valor esta na ultima posicao ou quando e inexistente.

d) Como o aumento das dimensoes influencia as operacoes?

O numero maximo de comparacoes cresce proporcionalmente ao total de elementos, $m \times n$.

e) Qual a complexidade?

$O(m \times n)$ no pior caso.

5. PARTE 4 - HANDS ON 1: INVESTIGACAO DO ARRAY

Foi utilizado um vetor com 10 temperaturas: **[19.0, 24.3, 21.3, 25.3, 25.6, 16.1, 15.2, 29.2, 19.4, 19.0]**. O programa percorre o vetor para calcular a media, identificar maior e menor valor, seus indices e a quantidade de valores acima da media.

```
temperaturas = [19.0, 24.3, 21.3, 25.3, 25.6,
                16.1, 15.2, 29.2, 19.4, 19.0]

soma = 0.0
for t in temperaturas:
    soma += t
media = soma / len(temperaturas)

maior = menor = temperaturas[0]
idx_maior = idx_menor = 0
for i in range(1, len(temperaturas)):
    if temperaturas[i] > maior:
        maior, idx_maior = temperaturas[i], i
    if temperaturas[i] < menor:
        menor, idx_menor = temperaturas[i], i

acima_media = sum(1 for t in temperaturas if t > media)
```

Grandeza	Resultado
Media	21,44 °C
Maior valor	29,2 °C - indice 7
Menor valor	15,2 °C - indice 6
Valores acima da media	4
Operacoes de percurso	aprox. 29 (10 + 9 + 10)
Complexidade	O(n) - linear

Como os percursos sao sequenciais e em quantidade fixa, a complexidade total permanece O(n).

6. PARTE 5 - HANDS ON 2: MONITORAMENTO DE SENSORES

A matriz possui 5 linhas (sensores) e 24 colunas (horarios), totalizando 120 medicoes. O indice **i** identifica o sensor e o indice **j** identifica o horario.

```
# sensores[i][j]: sensor i, hora j
medias_sensores = []

for i in range(5):
    soma = 0.0
    for j in range(24):
        soma += sensores[i][j]
    medias_sensores.append(soma / 24)

maior_valor = sensores[0][0]
sensor_maior = hora_maior = 0

for i in range(5):
    for j in range(24):
        if sensores[i][j] > maior_valor:
            maior_valor = sensores[i][j]
            sensor_maior, hora_maior = i, j

soma_total = sum(sum(linha) for linha in sensores)
media_geral = soma_total / 120

limite = 28.0
qtd_acima = sum(1 for linha in sensores for v in linha if v > limite)
```

Sensor	Media das 24 medicoes
Sensor 0	24,69 °C
Sensor 1	25,09 °C
Sensor 2	24,31 °C
Sensor 3	24,02 °C
Sensor 4	25,48 °C

Grandeza	Resultado
Maior temperatura	35,0 °C
Sensor responsavel	Sensor 1
Horario	12h
Media geral	24,72 °C
Limite informado	28,0 °C
Leituras acima do limite	38

Por que loops aninhados? Porque a estrutura possui duas dimensoes. Um loop percorre os sensores e outro percorre os 24 horarios. Uma varredura completa visita $5 \times 24 = 120$ posicoes, com complexidade $O(m \times n)$.

7. PARTE 6 - ANALISE E CONCLUSAO

1. O aumento do tamanho da estrutura influencia a quantidade de operacoes?

Sim. Quanto maior a entrada, maior a quantidade de operacoes. A taxa desse crescimento depende da complexidade do algoritmo.

2. Bubble Sort e Quick Sort crescem da mesma maneira?

Nao. O Bubble Sort cresce quadraticamente, $O(n^2)$, enquanto o Quick Sort apresenta $O(n \log n)$ no caso medio. Por isso, a diferenca aumenta bastante em entradas maiores.

3. Por que o resultado final nao e suficiente para comparar algoritmos?

Porque ambos podem produzir exatamente o mesmo vetor ordenado. A eficiencia aparece quando observamos comparacoes, trocas, movimentacoes e percursos necessarios para chegar ao resultado.

Conclusao geral

Os experimentos confirmam que a escolha do algoritmo e da estrutura de dados influencia diretamente o custo computacional. O Bubble Sort e simples e util para fins didaticos, mas sua complexidade $O(n^2)$ limita seu uso em grandes volumes. O Quick Sort apresenta comportamento medio $O(n \log n)$ e escala melhor, embora possa atingir $O(n^2)$ em particoes muito desbalanceadas. Nas matrizes, a busca sequencial e os percursos completos dependem do produto entre linhas e colunas, $O(m \times n)$.

8. REFERENCIAS BIBLIOGRAFICAS

CORMEN, Thomas H. et al. *Introduction to Algorithms*. 4th ed. MIT Press, 2022.

KNUTH, Donald E. *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2nd ed. Addison-Wesley, 1998.